POLITECNICO
MILANO 1863

Ingegneria del software

Design pattern comportamentali

Command

Motivazioni

- Consideriamo di voler creare una classe Menu per eseguire operazioni all'interno di una applicazione
- Una istanza di Menu contiene un insieme di MenuItem, uno per ciascuna azione da eseguire.
- Una istanza di Menu non deve conoscere i dettagli dell'applicazione associata, né delle azioni associate ai MenuItem

Motivazioni

- **Problema:** gestire un insieme di comandi non conosciuti a priori
- Istanze di Menu non devono avere dipendenze con i comandi
- Ipotizziamo anche che la azione associata ad un MenuItem può cambiare a runtime.

Una soluzione non ideale

```
class Menu {

        MyApplicationDocument document;

        public Menu(MyApplicationDocument document){
                this.document = document;
        }

        public void copy(){ … }
        public void paste() { … }
        public void new() { … }
}
```

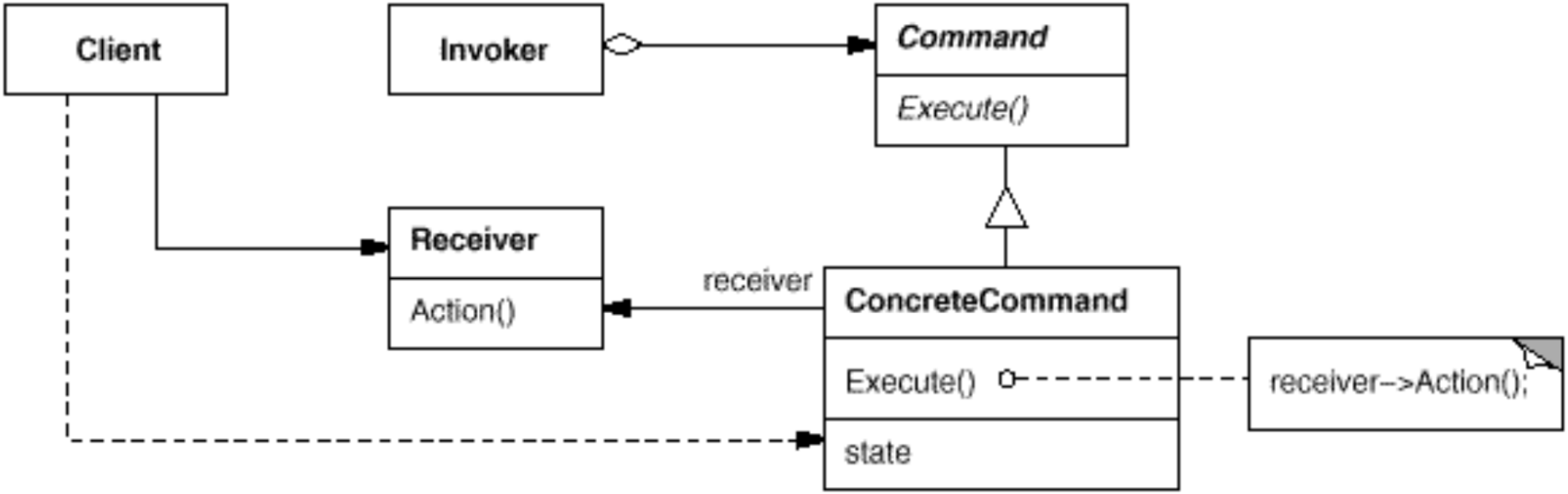
Una soluzione non ideale (2)

```
class Menu {
        List<MenuItem> items = new ArrayList<>();
        public void addMenuItem(MenuItem a){ … }
}

class MenuItem {

        MyApplicationDocument document;

        public void clicked(){ … }

}
```

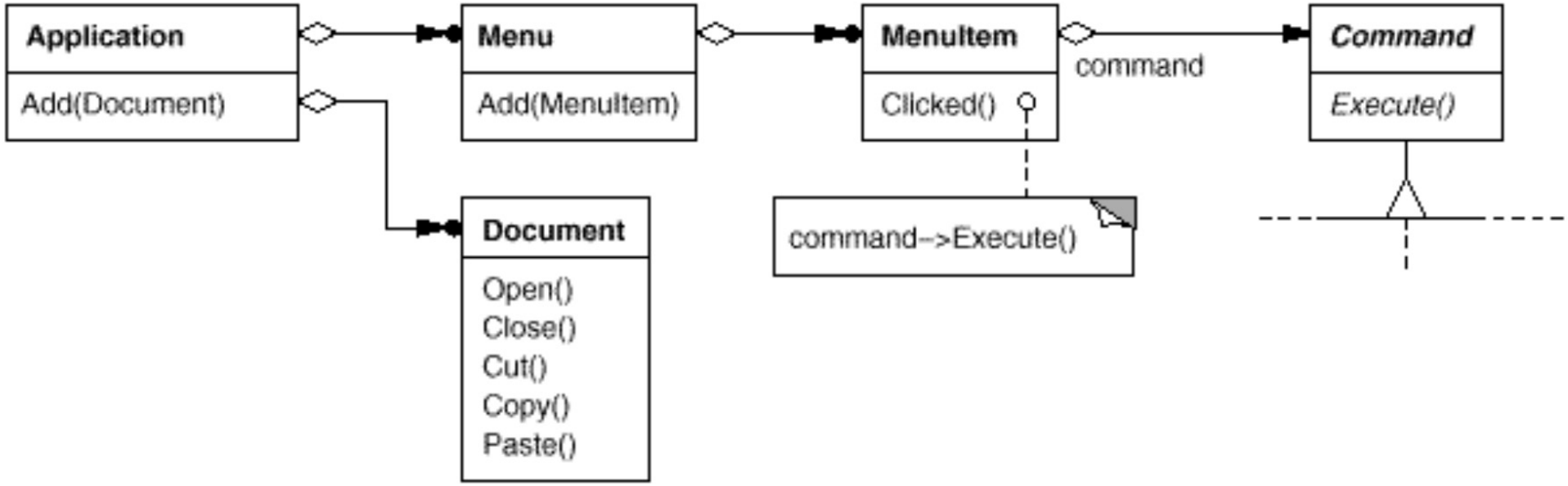
Obbiettivi

- Definire una interfaccia Command
- La struttura più semplice di Command è che contenga un solo metodo «execute»
- MenuItem ha un riferimento a un oggetto di tipo Command
- La azione da compiere diventa slegata dagli oggetti che la usano

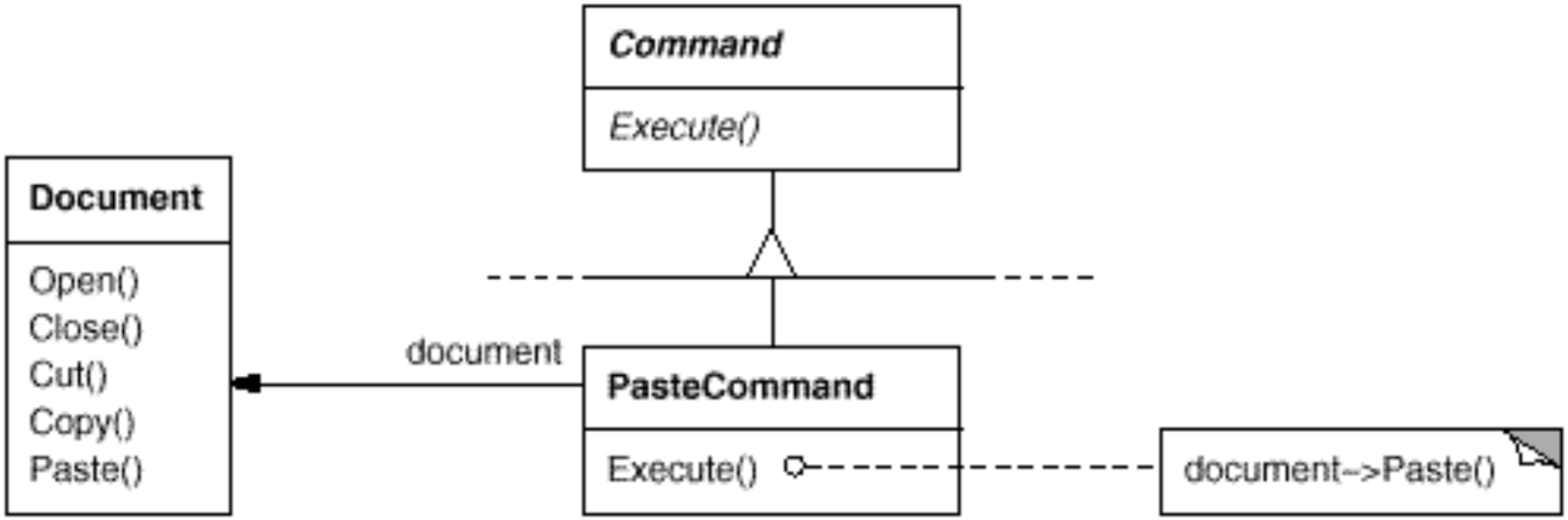
Struttura


Client
Invoker
Command
Execute()
Receiver
Action()
receiver
ConcreteCommand
Execute()
state
receiver->Action();

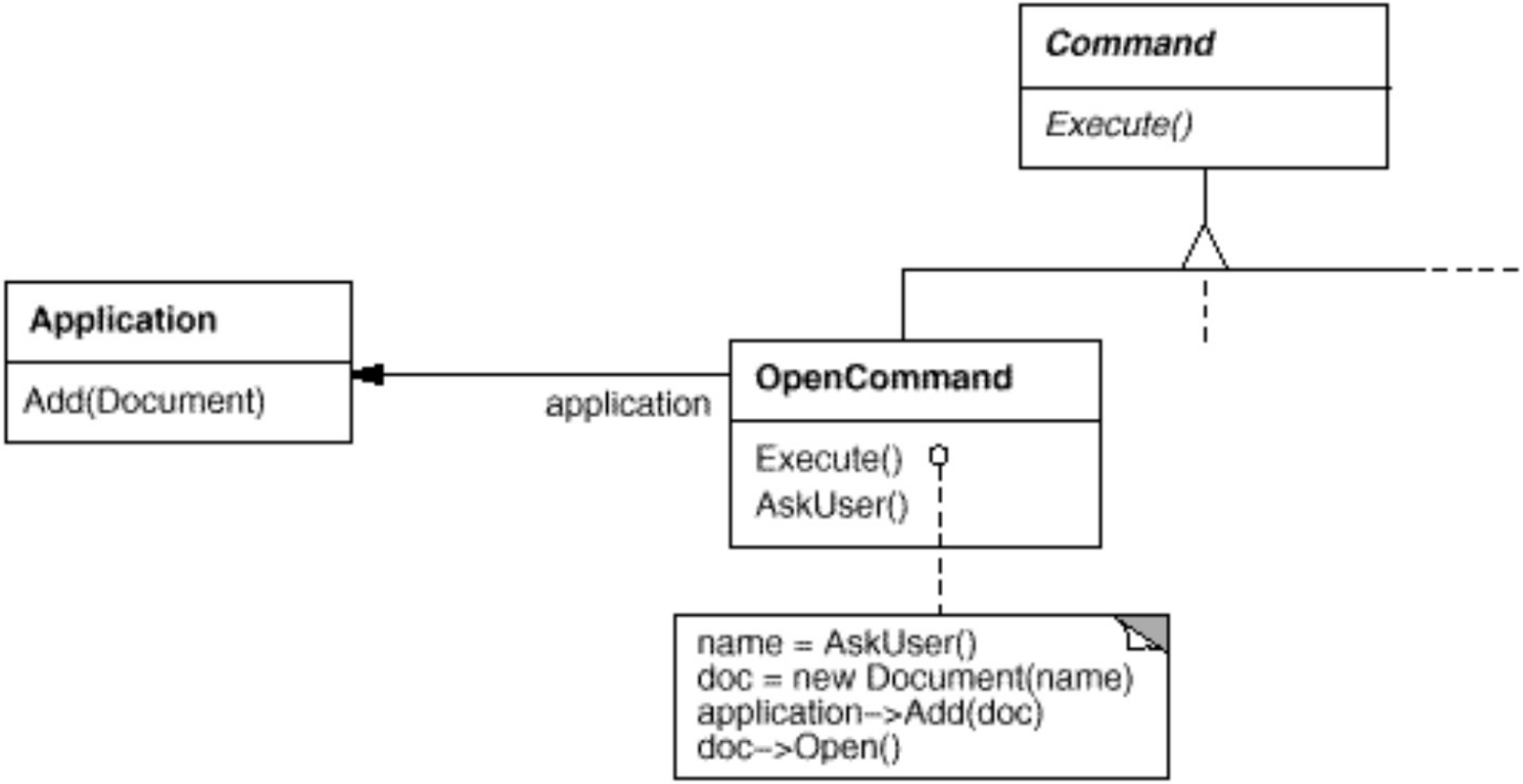
Esempio


Application
Add(Document)
Menu
Add(MenuItem)
MenuItem
Clicked()
command
Command
Execute()
Document
Open()
Close()
Cut()
Copy()
Paste()
command->Execute()

Esempio


Command
Execute()
Document
Open()
Close()
Cut()
Copy()
Paste()
document
PasteCommand
Execute()
document->Paste()

Esempio


Command
Execute()
Application
Add(Document)
application
OpenCommand
Execute()
AskUser()
name = AskUser()
doc = new Document(name)
application->Add(doc)
doc->Open()

Esempio

```
interface Command {
      void execute();
      …
}


class PasteCommand implements Command {
      private Document doc;
      public PasteCommand(Document doc){ this.doc = doc; }
      public void execute(){
        …
      }
}
```

Esempio

```
class Menu {
      List<MenuItem> items = new ArrayList<>();
      public void addMenuItem(MenuItem a){ … }
}

class MenuItem {

      Command command;

      public void setCommand(Command cmd){ this.command = cmd }

      public void clicked(){
            command.execute();
      }

}
```